# Collective Operations in Distributed Machine Learning

A Focus on Large Language Model (LLM) Training

Chen Avin

# Introduction to Collective Operations

- Collective operations are mechanisms for coordinating data sharing and processing across devices.
- Critical for efficient scaling in distributed machine learning.
- Used extensively in LLM training to handle massive computational workloads.
- Focus on seven key collectives: Reduce, Gather, All-Reduce, All-Gather, Scatter, Reduce-Scatter, All-to-All.

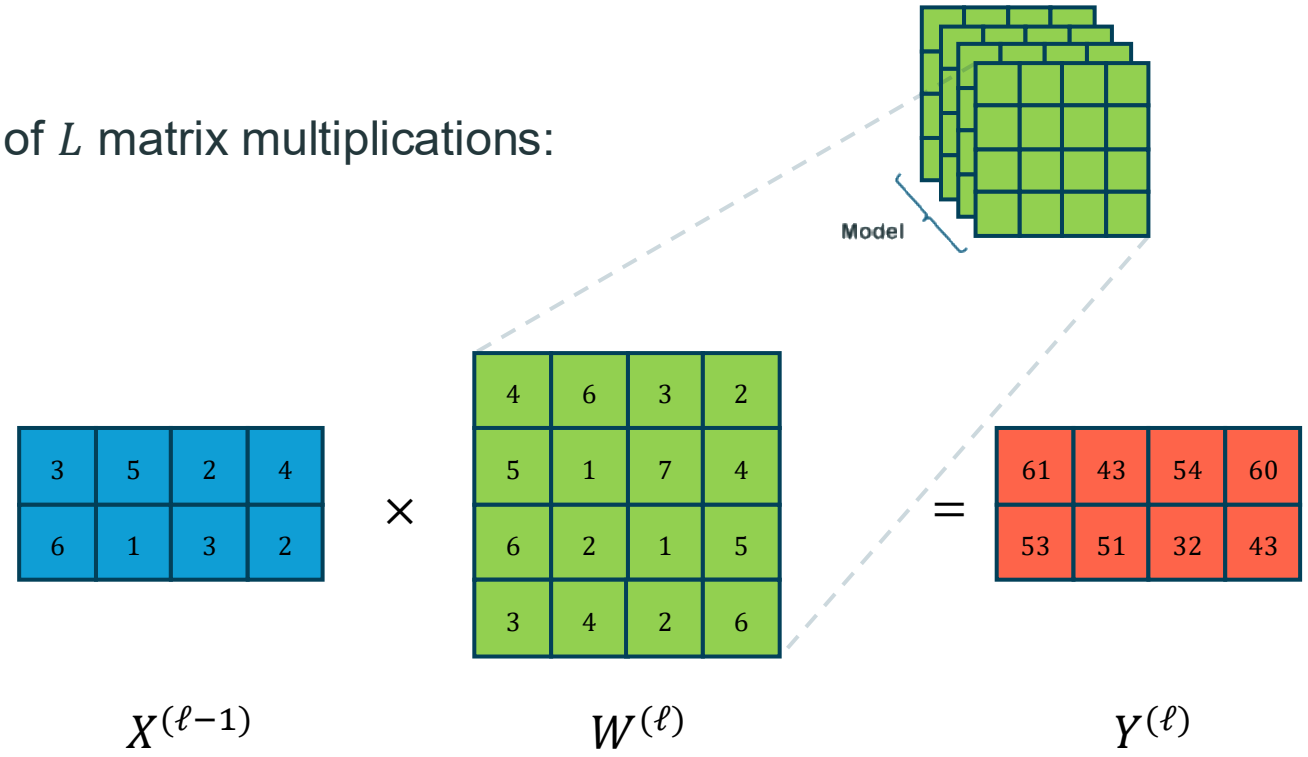
# Distributed Machine Learning

https://huggingface.co/spaces/HoangPhuc7678/ultrascale-playbook?section=high_level_overview

T3P: Topology-Tailored Tensor Parallelism. SB Yochana, C Avin, G Scalosub
Proceedings of the 2nd Workshop on Networks for AI Computing, 81-88

# DNN Model Architecture (Simplified)

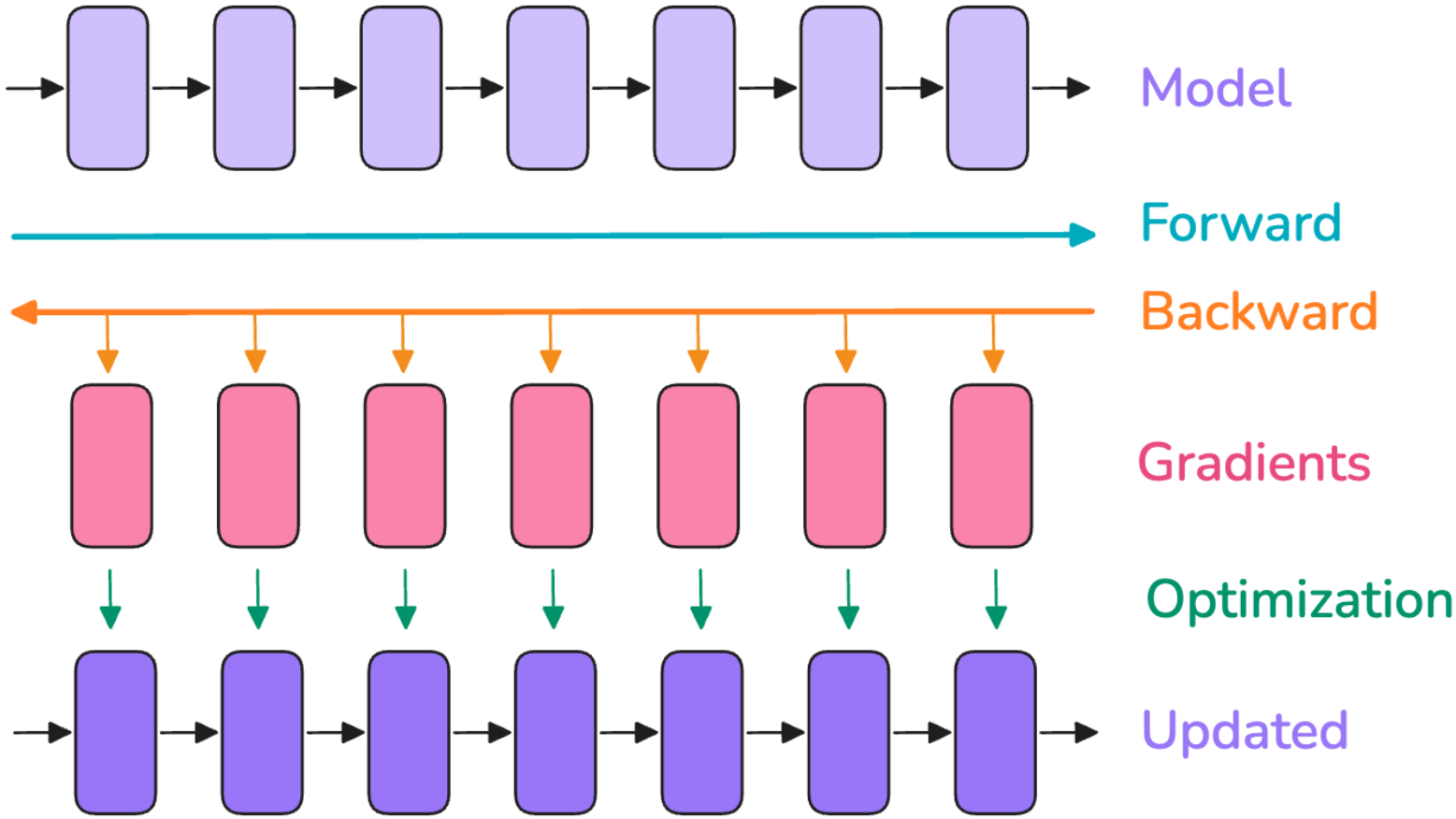
- A model consisting of $L$ matrix multiplications:



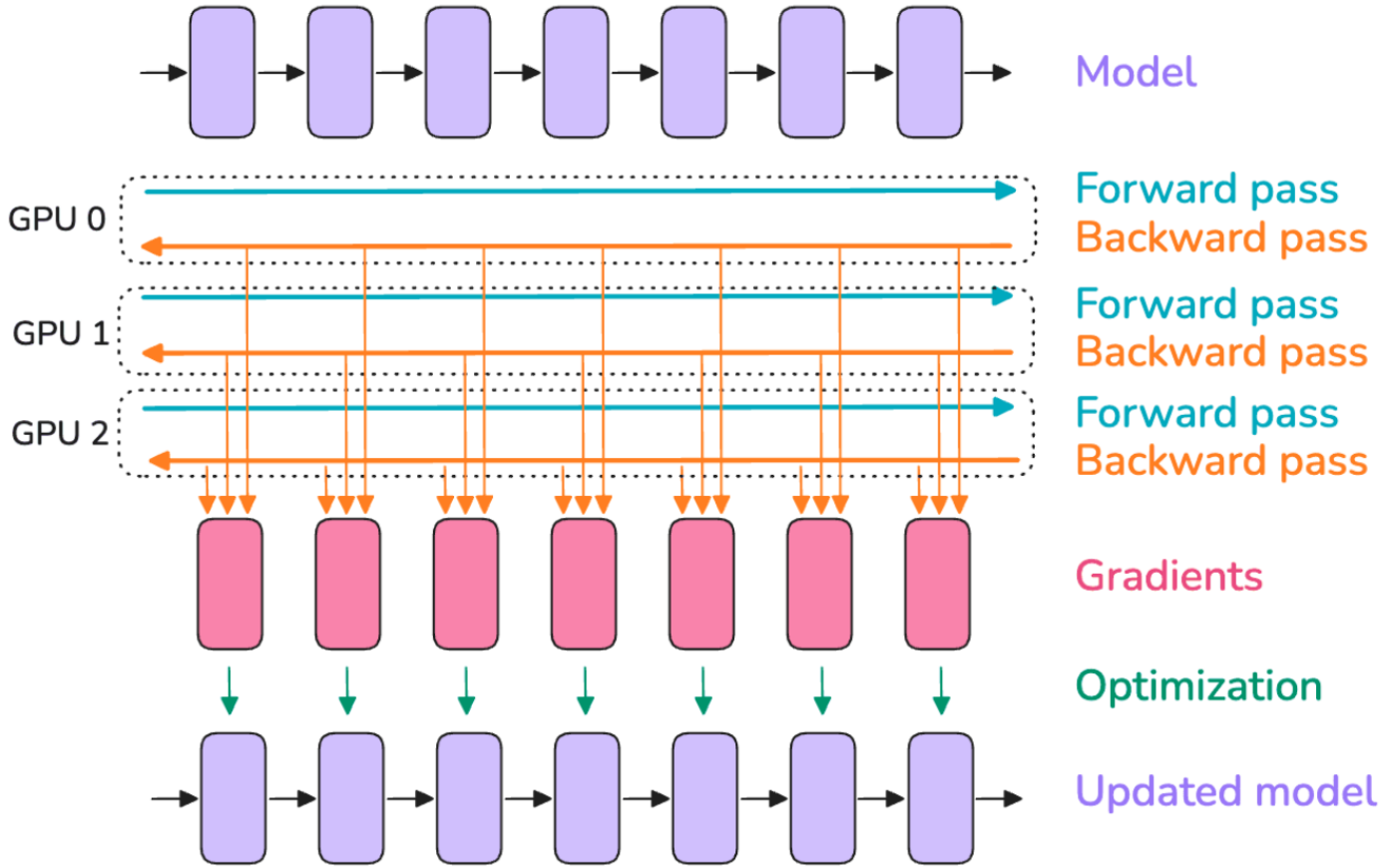

- Input: $X^{(0)} \in \mathbb{R}^{n_x \times n_0}$
- For each layer $\ell$: $W^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_\ell}$
- Activation function $f(\cdot)$ is applied element-wise

$$Y^{(\ell)} = f\left(X^{(\ell-1)} W^{(\ell)}\right), \qquad \ell \in \{1, \ldots, L\}$$

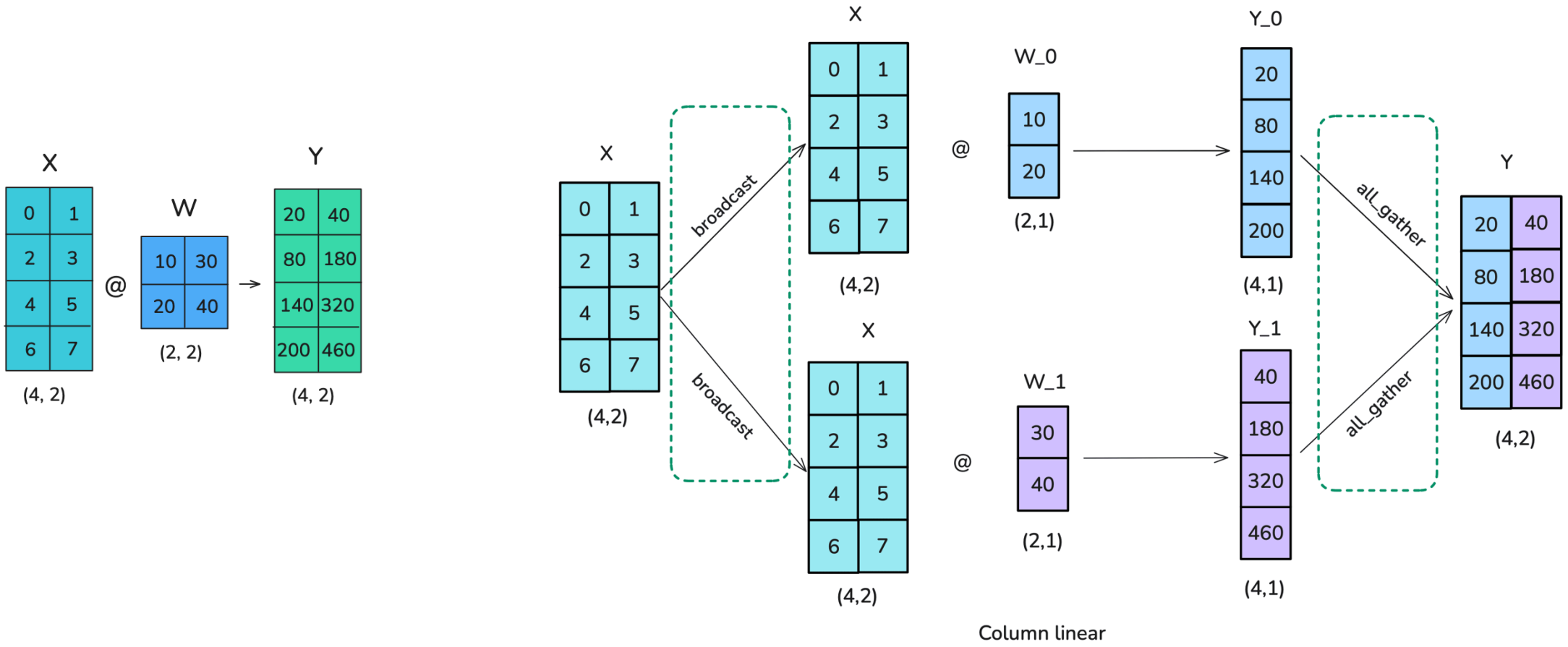
# Training on one GPU



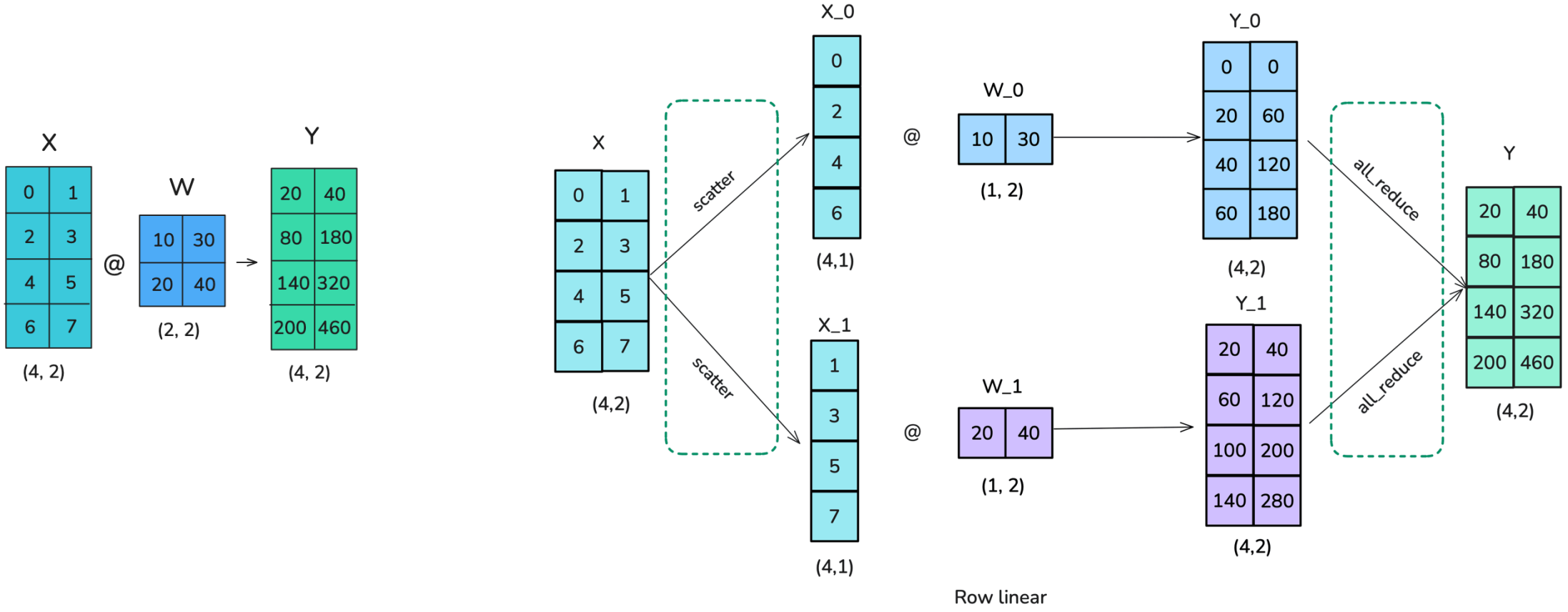
# Data Parallelism



# Tensor Parallelism (column)



# Tensor Parallelism (row)



# Sharding

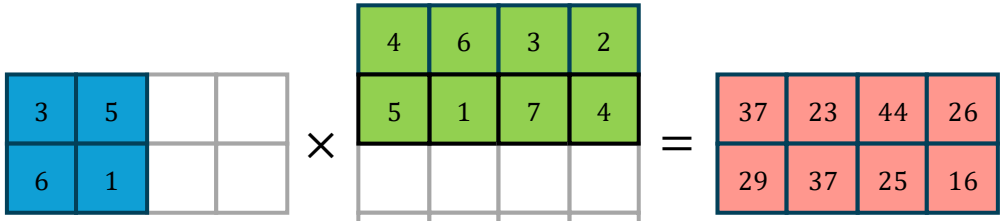
- **Sharding:** Dividing a tensor into shards
- **1D TP:** Partition a weight matrix along one dimension



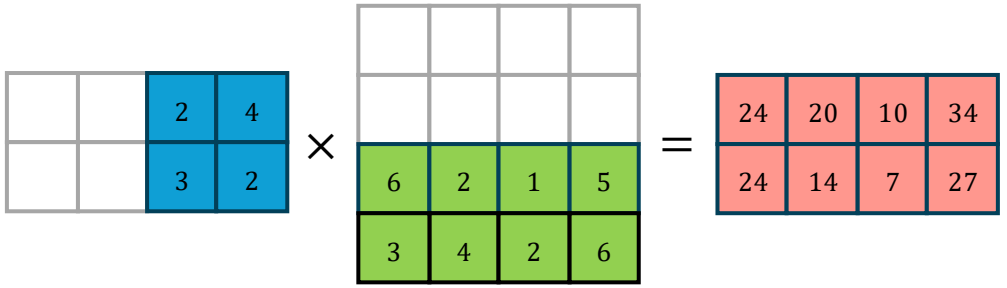

## Row-wise

$X^{(\ell-1)}$ Shard(1) × $W^{(\ell)}$ Shard(0) = $Y^{(\ell)}$ Partial(SUM)

Device 1



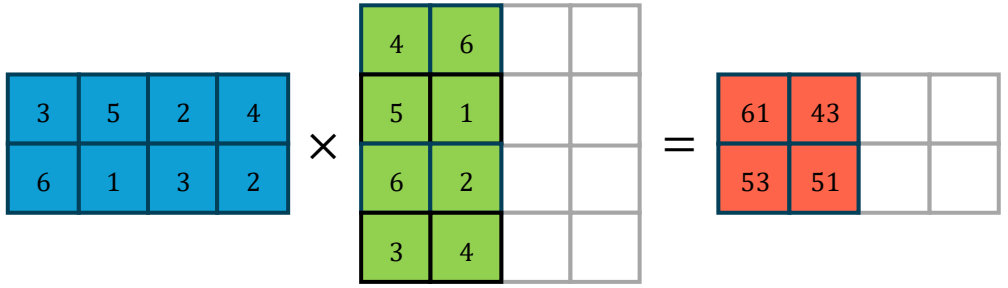

Device 2



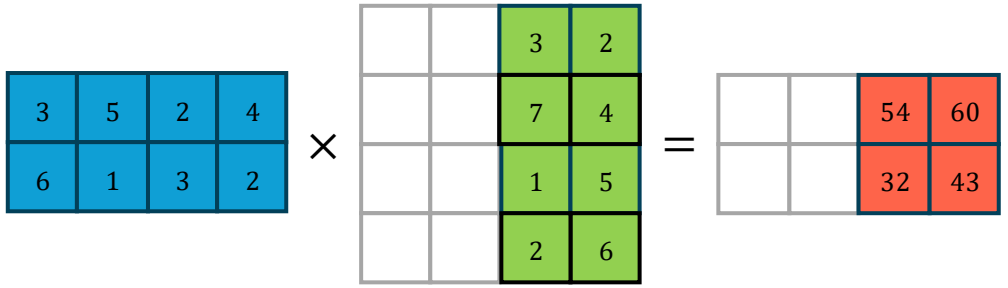

## Column-wise

$X^{(\ell-1)}$ Replicate × $W^{(\ell)}$ Shard(1) = $Y^{(\ell)}$ Shard(1)

Device 1

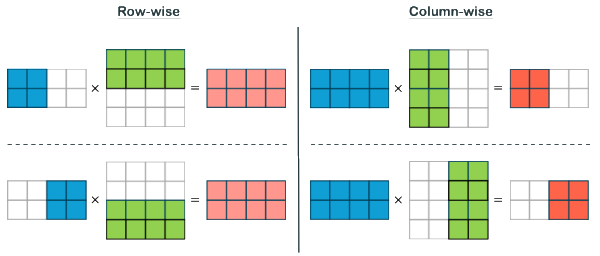


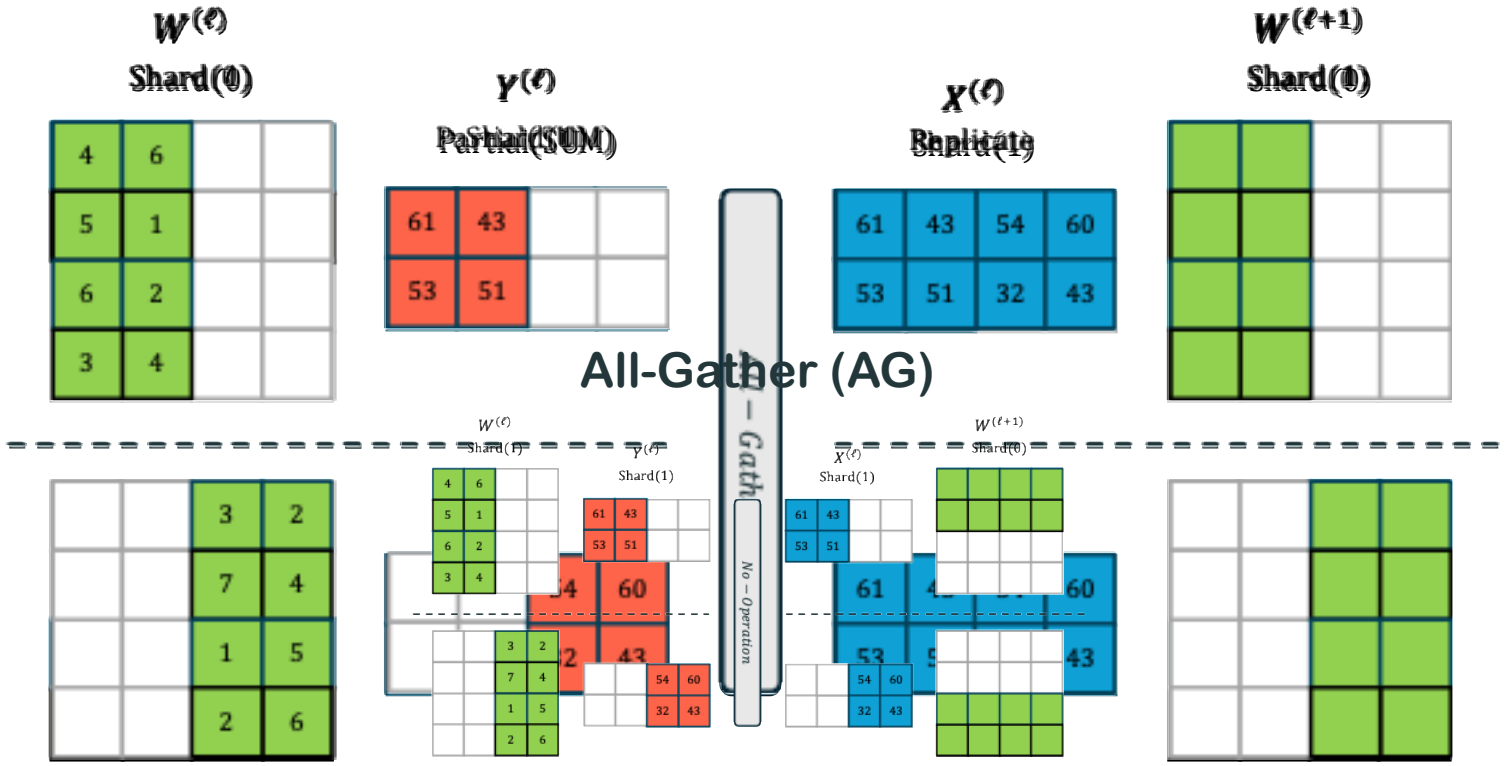

Device 2



# Resharding & Collective Operations

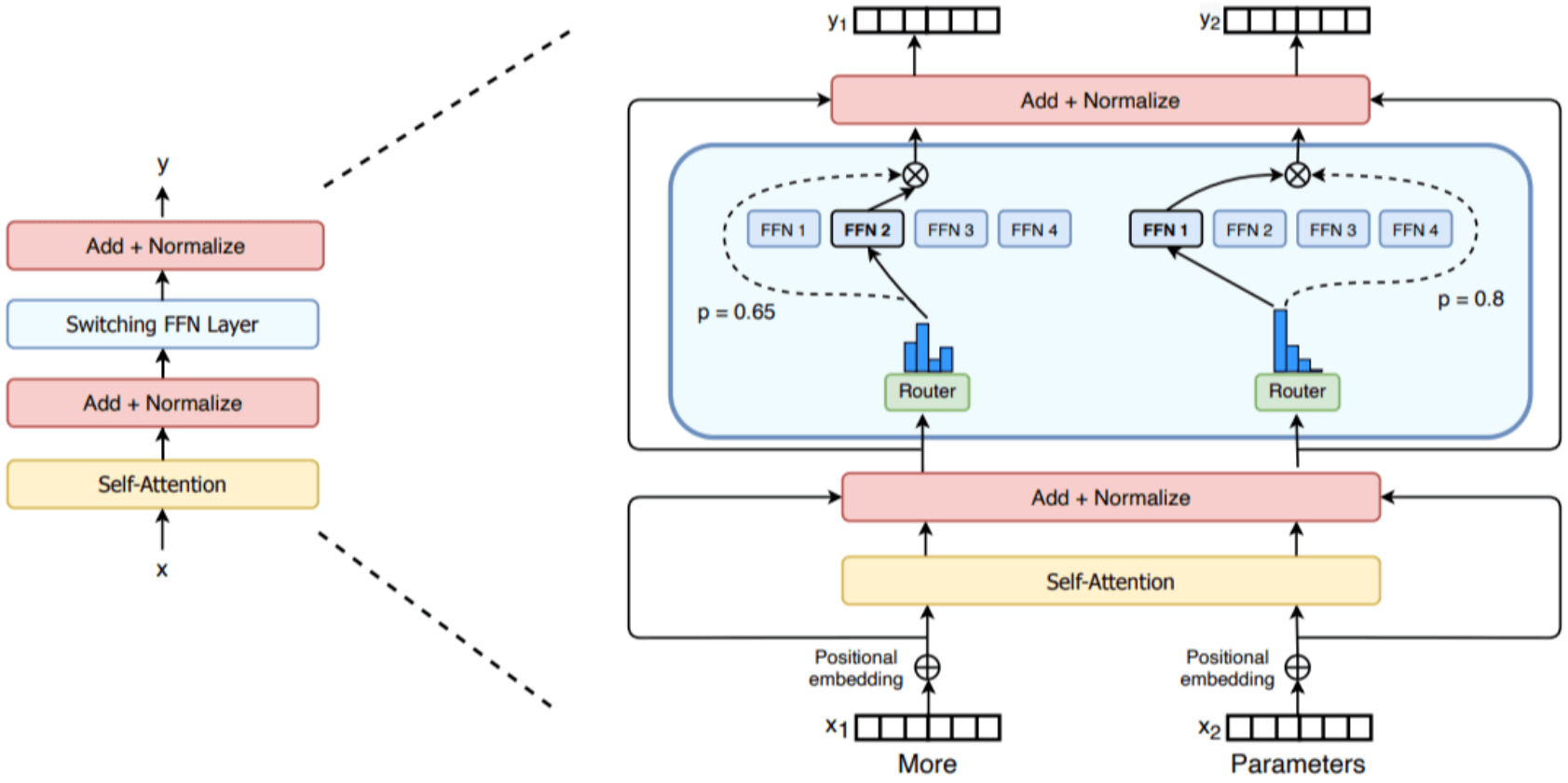
- Consecutive layers may require **re-sharding:**
  - Converts the output sharding of layer $\ell$ to the input sharding of layer $\ell + 1$
  - Performed with collective operations.



# Pipeline Parallelism

# Mixture of Expert



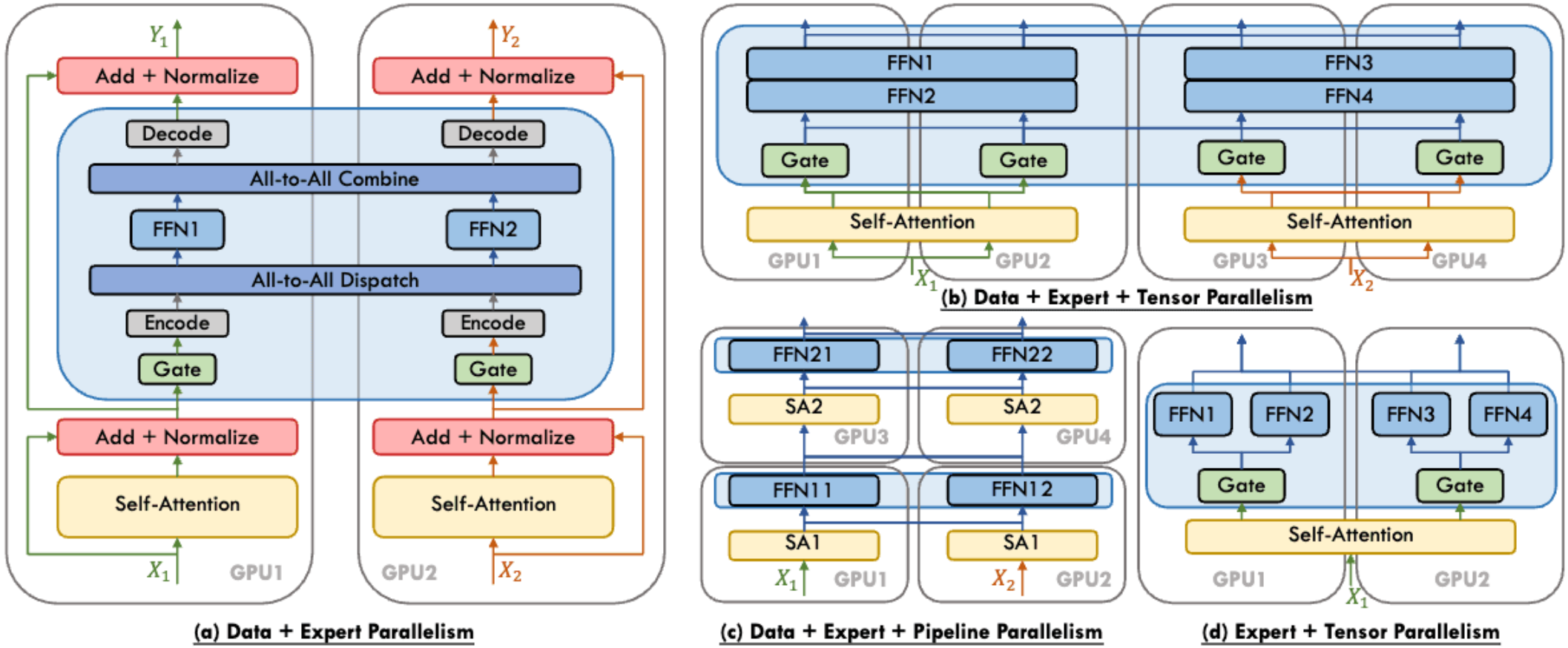


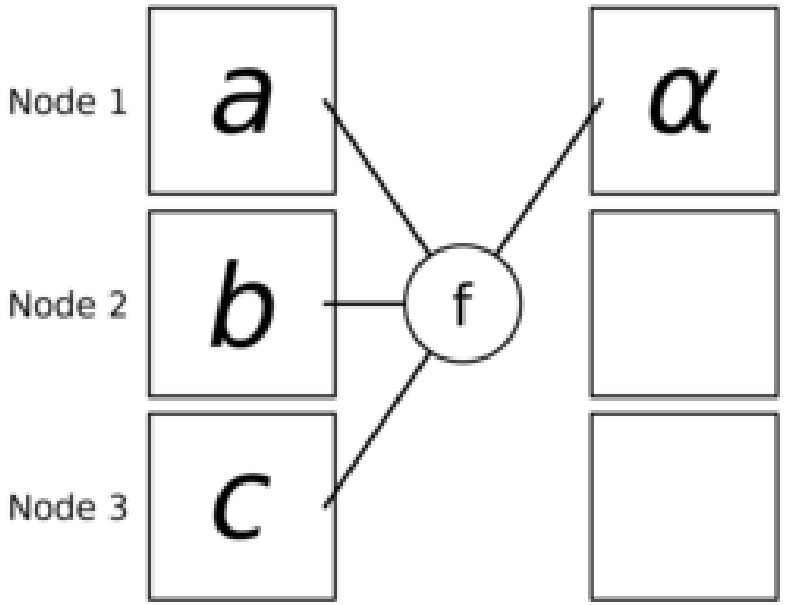

Fig. 8. Schematic depiction of diverse parallel strategies for MoE. For clarity and conciseness, this illustration omits some All-to-All, All-Reduce, Point-to-Point communication within parallelism, and Normalization, Encode, Decode, Gate in subfigures (b), (c), and (d).

# Overview of Collectives

- Reduce: Aggregates data across devices to a single device.
- Gather: Combines data from all devices into one.
- All-Reduce: Combines data and distributes results to all devices.
- All-Gather: Collects all data from devices and shares it back with all.
- Scatter: Divides data and sends portions to devices.
- Reduce-Scatter: Combines and distributes data in a single operation.
- All-to-All: Each device sends data to all other devices.

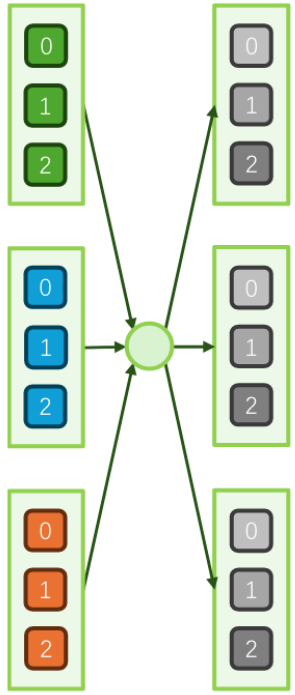
# Reduce

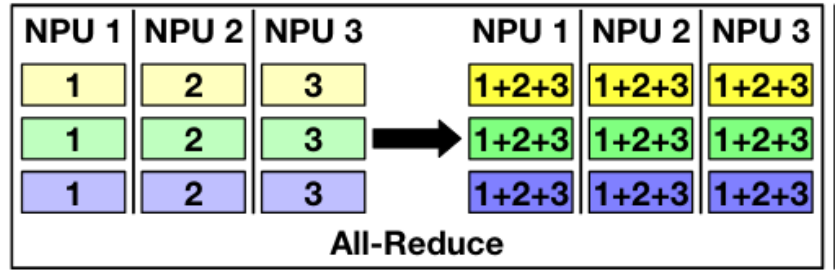
- **What It Does**:
  - Aggregates data from all devices and places the result on a single device. Typical operations include summation, average, or max.
- **Use in LLM Training**:
  - **Parallelism Type**: Data Parallelism.
  - **Scenario**: Aggregating the computed loss values from all devices to one device for logging or checkpointing purposes.
  - Example: When training with large batches distributed across GPUs, loss values are reduced to calculate metrics like perplexity



# All-Reduce

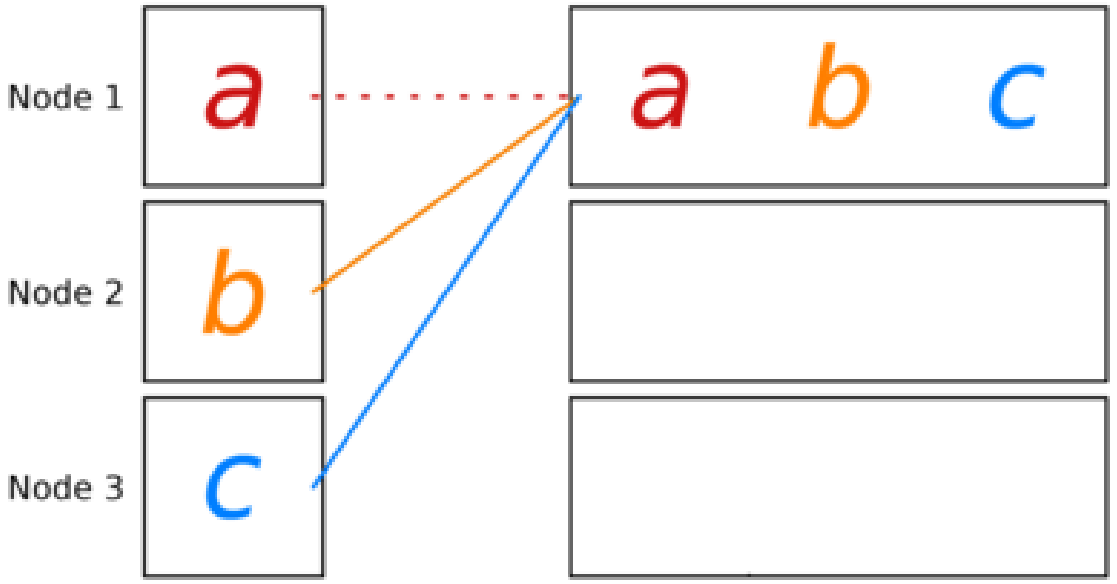
- **What It Does**:
  - Combines data across all devices (e.g., summing gradients) and distributes the combined result back to all devices.
- **Use in LLM Training**:
  - **Parallelism Type**: Data Parallelism.
  - **Scenario**: Gradient synchronization during backpropagation. After computing local gradients, all-reduce ensures all GPUs have the same gradient values for consistent weight updates.
  - Example: In GPT-like models trained with data parallelism, gradients computed locally are all-reduced across devices to synchronize updates.



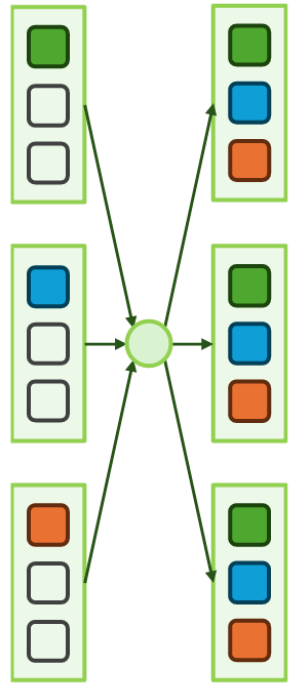
# Gather

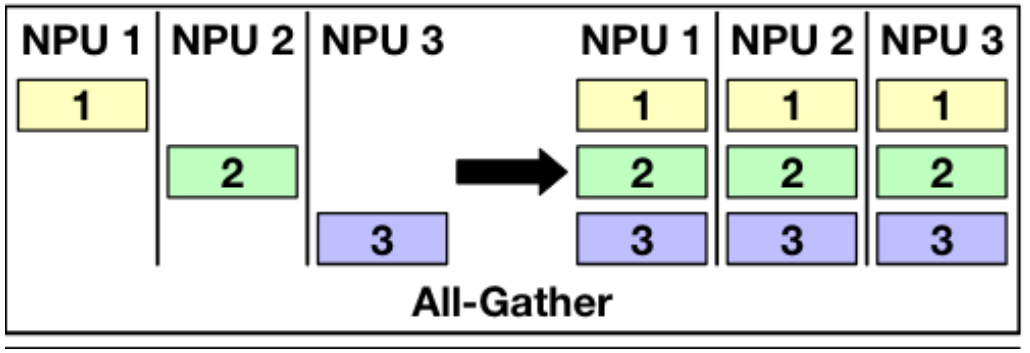
- **What It Does**:
  - Collects data from all devices into one without performing any computation.
- **Use in LLM Training**:
  - **Parallelism Type**: Model Parallelism.
  - **Scenario**: Consolidating embedding vectors calculated independently on multiple GPUs into a single batch for post-processing (e.g., input to a fully connected layer).
  - Example: Gathering partial outputs of transformers computed on separate GPUs into a unified tensor.



# All-Gather

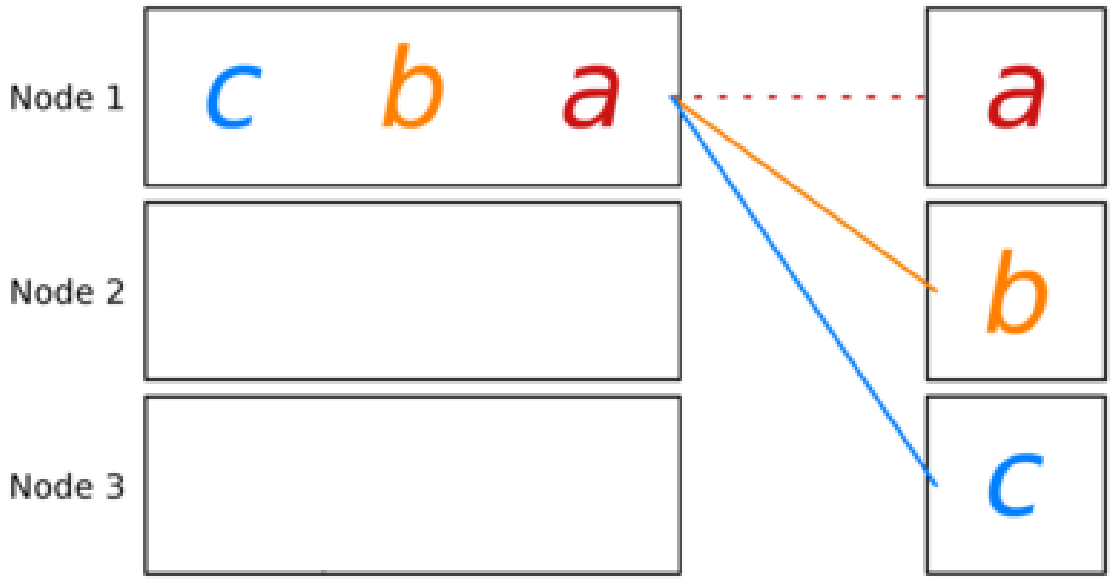
- **What It Does**:
  - Each device collects and shares its local data with all other devices.
- **Use in LLM Training**:
  - **Parallelism Type**: Model Parallelism or Expert Parallelism.
  - **Scenario**: Sharing activation outputs across GPUs for forward or backward passes in model-parallel architectures.
  - Example: When splitting layers of a transformer across GPUs, activations from one GPU must be gathered by all others to continue computation.





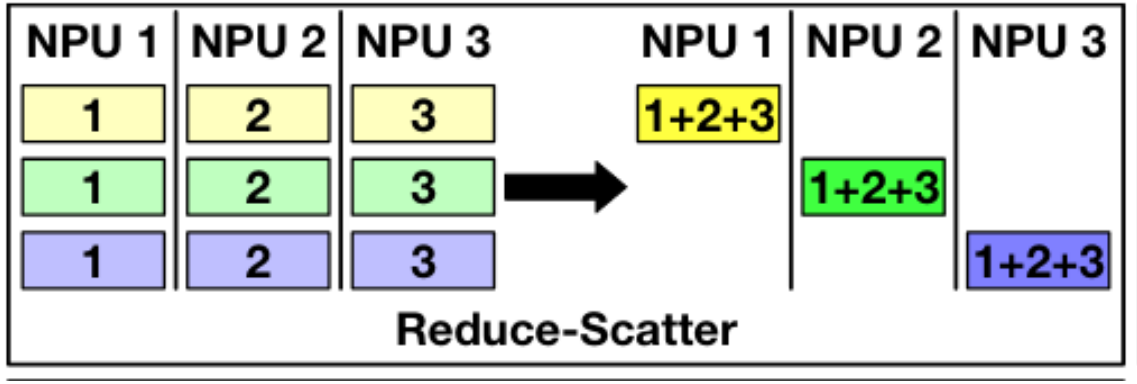
# Scatter

- **What It Does**:
  - Divides a dataset into smaller parts and sends each part to a different device.
- **Use in LLM Training**:
  - **Parallelism Type**: Data Parallelism.
  - **Scenario**: Distributing input batches or datasets among GPUs for preprocessing or training.
  - Example: Sharding a large input batch into smaller chunks and scattering them to GPUs for parallel processing.



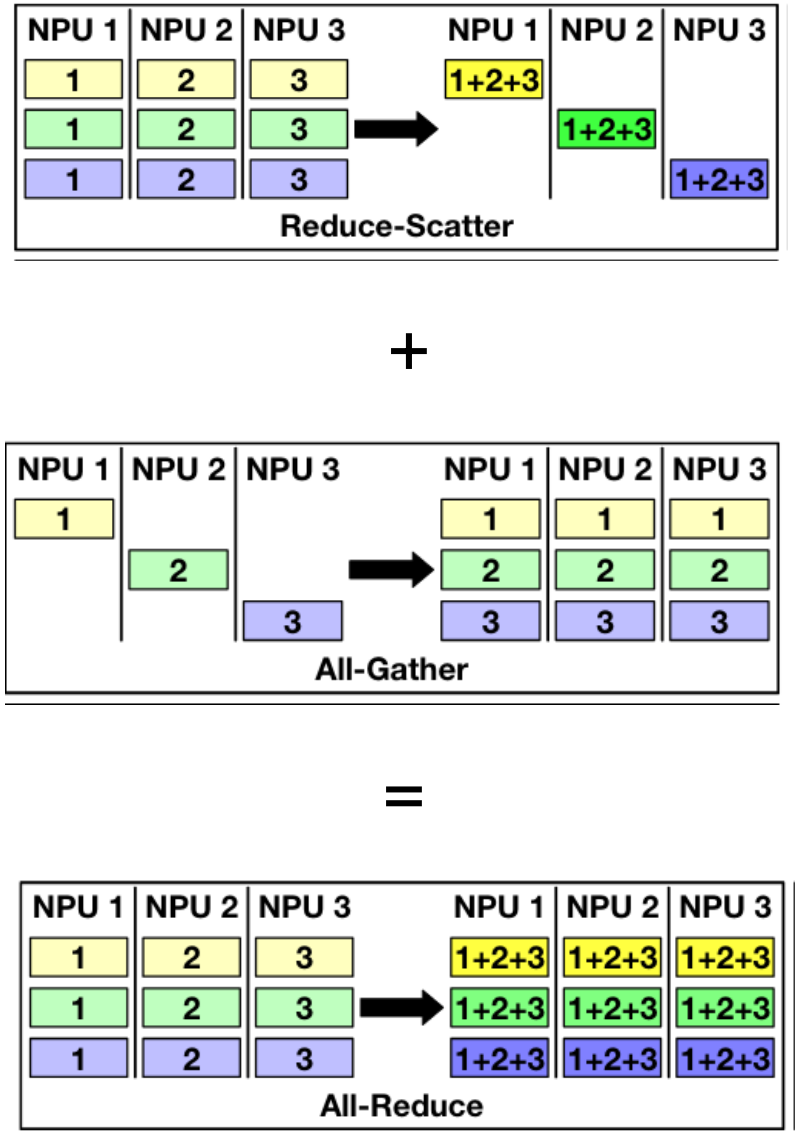
# Reduce-Scatter

- **What It Does**:
  - Combines data from all devices (e.g., sum or average) and simultaneously distributes portions of the result back to devices.
- **Use in LLM Training**:
  - **Parallelism Type**: Model Parallelism.
  - **Scenario**: Efficient gradient reduction and distribution in tensor-parallel training setups.
  - Example: When distributing a tensor's updates in transformer-based LLMs, gradients are reduced and scattered to ensure only the necessary updates are shared with each GPU.



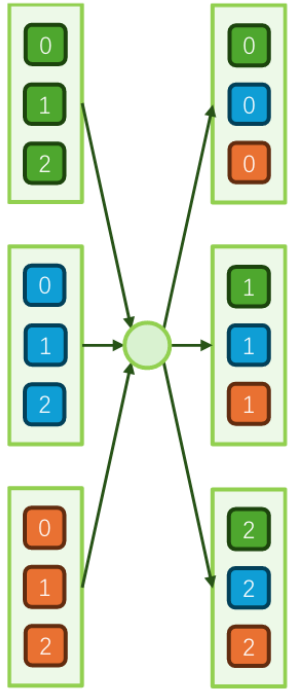
# Reduce-Scatter

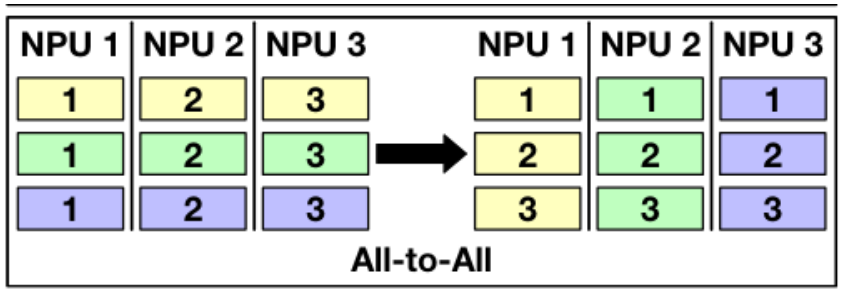
- We can do All-reduce using Reduce-Scatter



# All-to-All

- **What It Does**:
  - Each device sends its local data to all other devices.
  - Matrix Transpose
- **Use in LLM Training**:
  - **Parallelism Type**: Tensor Parallelism or Expert Parallelism.
  - **Scenario**: Exchanging intermediate results between devices during tensor-parallel or mixture-of-experts (MoE) layers.
  - Example: In MoE architectures, tokens assigned to specific experts are routed to corresponding GPUs using an all-to-all communication step.



# Summary of Collectives

- Recap of collectives:
  - Data Parallelism: Reduce, All-Reduce, Scatter.
  - Model Parallelism: Gather, All-Gather, Reduce-Scatter.
  - Expert Parallelism: All-to-All, All-Gather.
- Contribution to scalability and efficiency in LLM training.

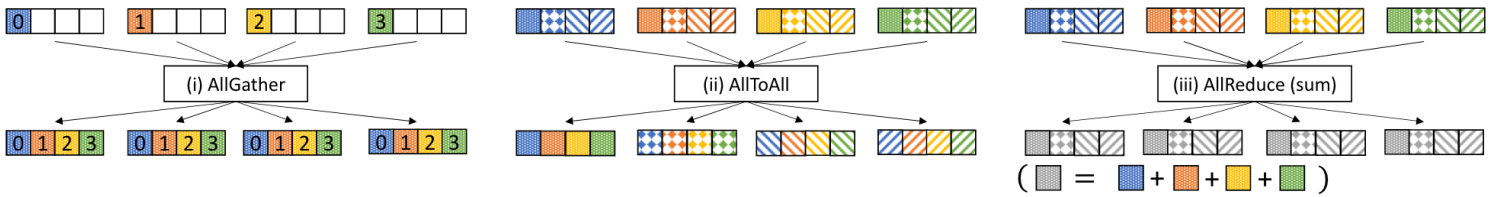



Figure 2: The initial and final data buffers on four GPUs participating in different collectives.

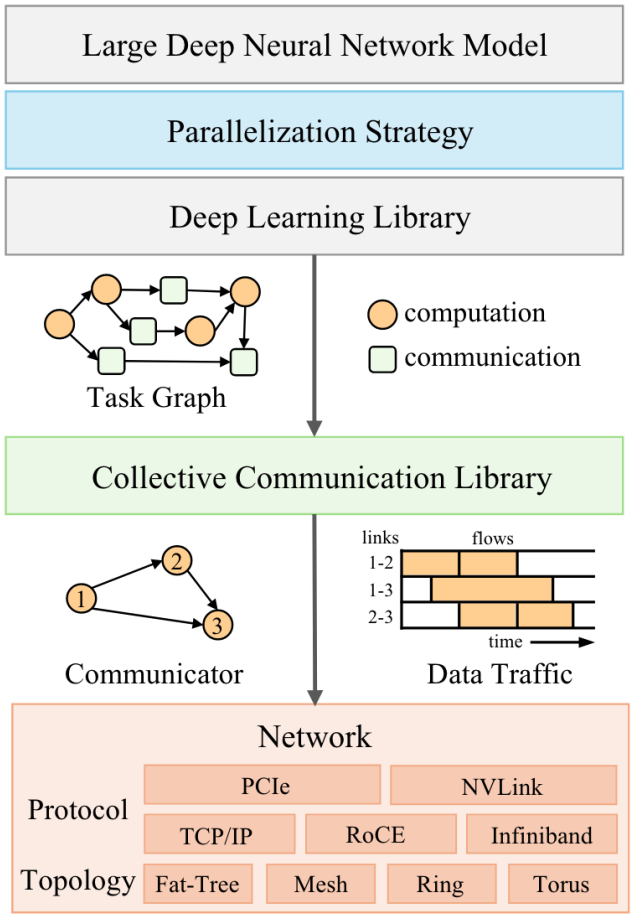



Fig. 1. Distributed training architecture from a communication perspective.

# Introduction to Algorithmic Implementation

- The performance of collective operations depends significantly on the algorithm used, especially when implemented over communication networks.
- The following topics will cover different algorithmic approaches for collective operations:
  - Complete Graph Model: Direct communication between all nodes.
  - Ring Algorithm (for All-Reduce)
  - Recursive Doubling (for All-Gather)
  - Bruck Algorithm: Optimizes data sharing for All-Gather in specific scenarios.
  - Recursive halving Algorithm: A method to reduce communication steps, optimizing for certain network topologies.

Reference paper: "Optimization of Collective Communication Operations in MPICH" by Thakur et al., 2005 (Link to Paper: https://web.cels.anl.gov/~thakur/papers/ijhpca-coll.pdf).

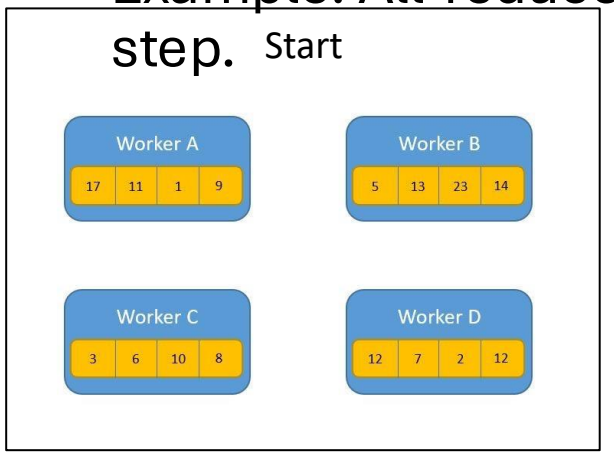
# Key Parameters in Collective Operations

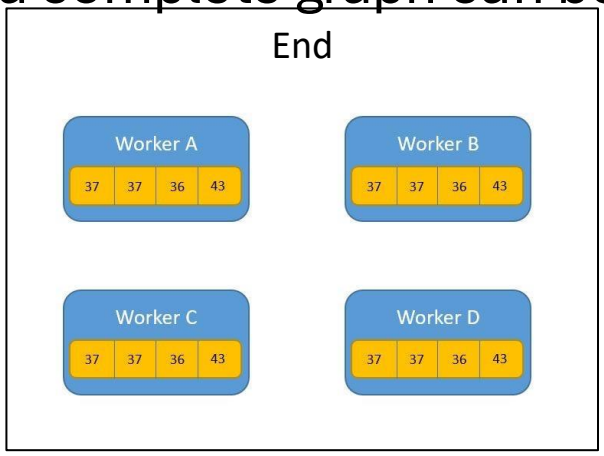
- $p$ (Number of Processes): Represents the number of nodes or processes involved in the collective operation.
  - Example: For an All-Reduce operation, p would be the total number of GPUs or nodes participating in the operation.
- $n$ (Data Size): Refers to the amount of data being communicated by each process.
  - Example: In All-Gather, n would represent the number of data elements or the size of the array each node wants to share.
- $\alpha$ (Latency per Message): The constant latency associated with sending a message between nodes.
  - Example: The time it takes to initiate and complete a communication is, independent of the data size.
- $\beta$ (Bandwidth per Message): The bandwidth available for sending data between nodes.
  - Example: The rate at which data can be transferred between nodes during a collective operation.

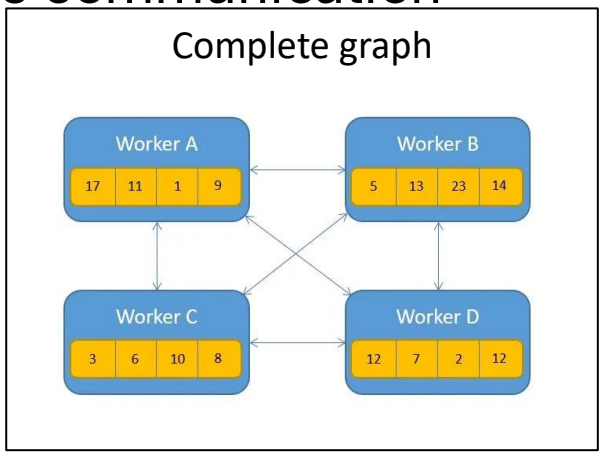
- The time complexity of collective operations can often be expressed in terms of these parameters. $T(p,n) = \alpha \cdot f(p) + \beta n \cdot g(p)$
- Latency Vs. Bandwidth

# Complete Graph Model

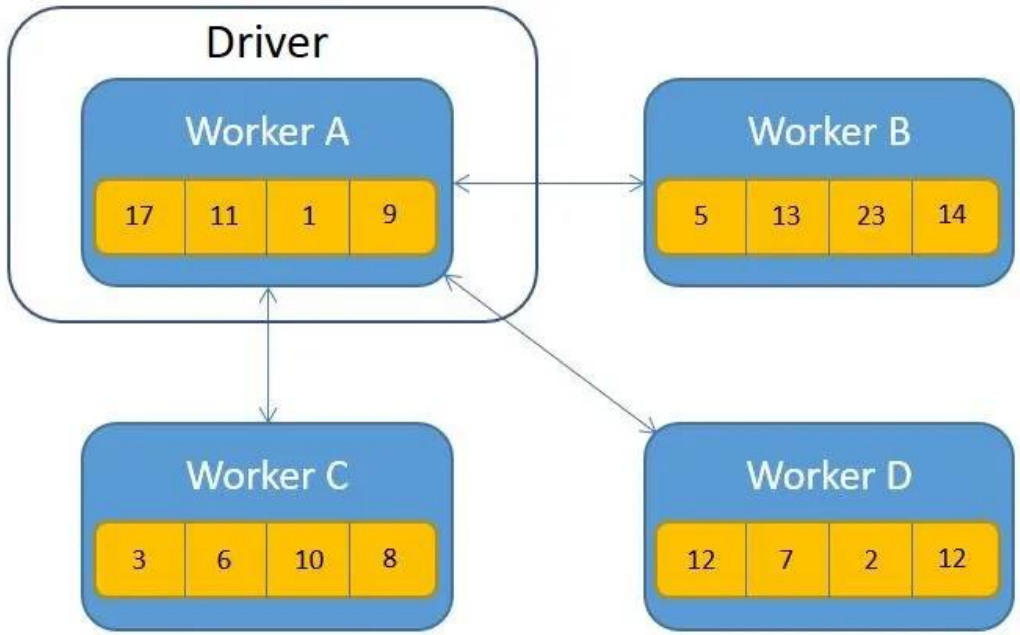
- Definition: In a complete graph, all nodes are directly connected to each other.
- Time Complexity: $O(\alpha + \beta n)$, as data can be shared in constant time between nodes.
- Message Complexity: $O(pn)$, where p is the number of nodes (since each node sends messages to all other nodes).
- Example: All-reduce in a complete graph can be done in a single communication step.

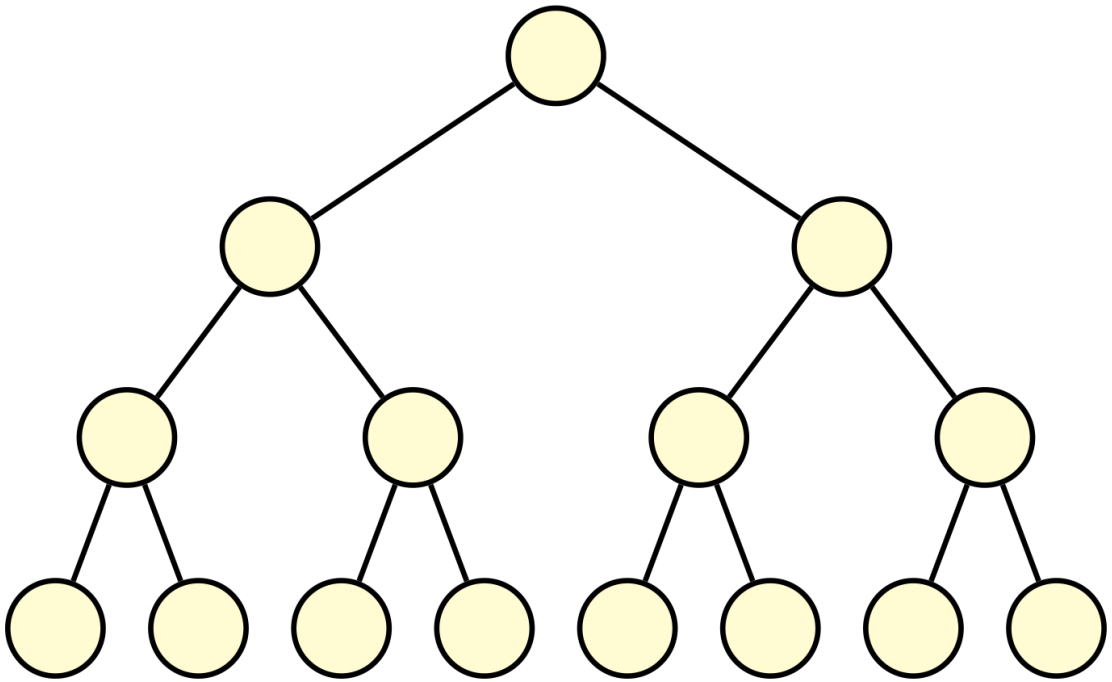


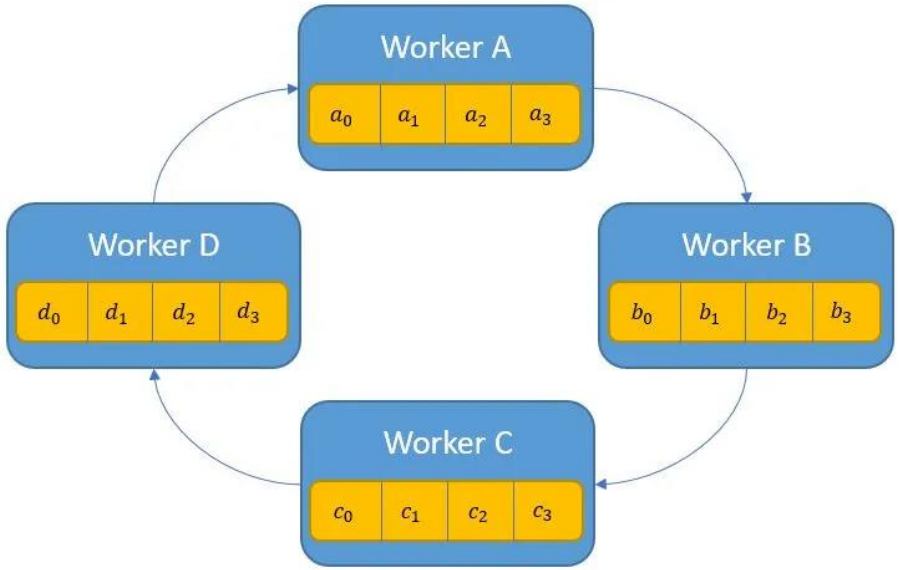
# Parameter Server



# Ring Algorithm for All-Reduce

- Description: The ring algorithm reduces data in a circular fashion where each node communicates with its neighbor.
- Ring Algorithm Phases (without pipeline):
  - Collect full information.
  - Compute.
- Time Complexity: $O((\alpha + n\beta)(p - 1))$
- Message/BW Complexity: $O(pn)$.

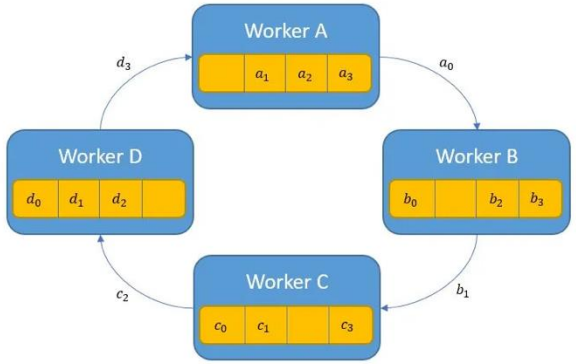


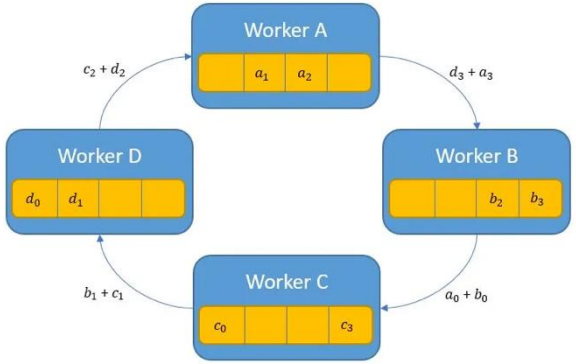
# Ring Algorithm for All-Reduce

- We can do better!
- Ring Algorithm Phases (with pipeline):
  - Reduce-Scatter Phase: Each node sends a portion of its data to the next node, reducing data along the way.
  - All-Gather Phase: Once data has been reduced, nodes gather the data from others to complete the operation.
- Time Complexity: $2(p\ -\ 1)\alpha\ +\ 2\frac{p-1}{p}n\beta$
- Message/BW Complexity: $O\left(\frac{p-1}{p}n\right)$. (optimal)

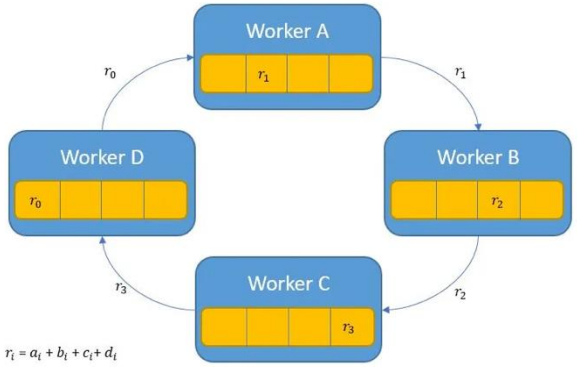
# Ring Algorithm for All-Reduce

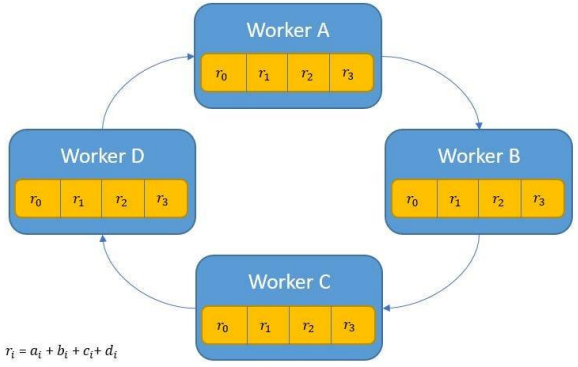
Reduce-Scatter

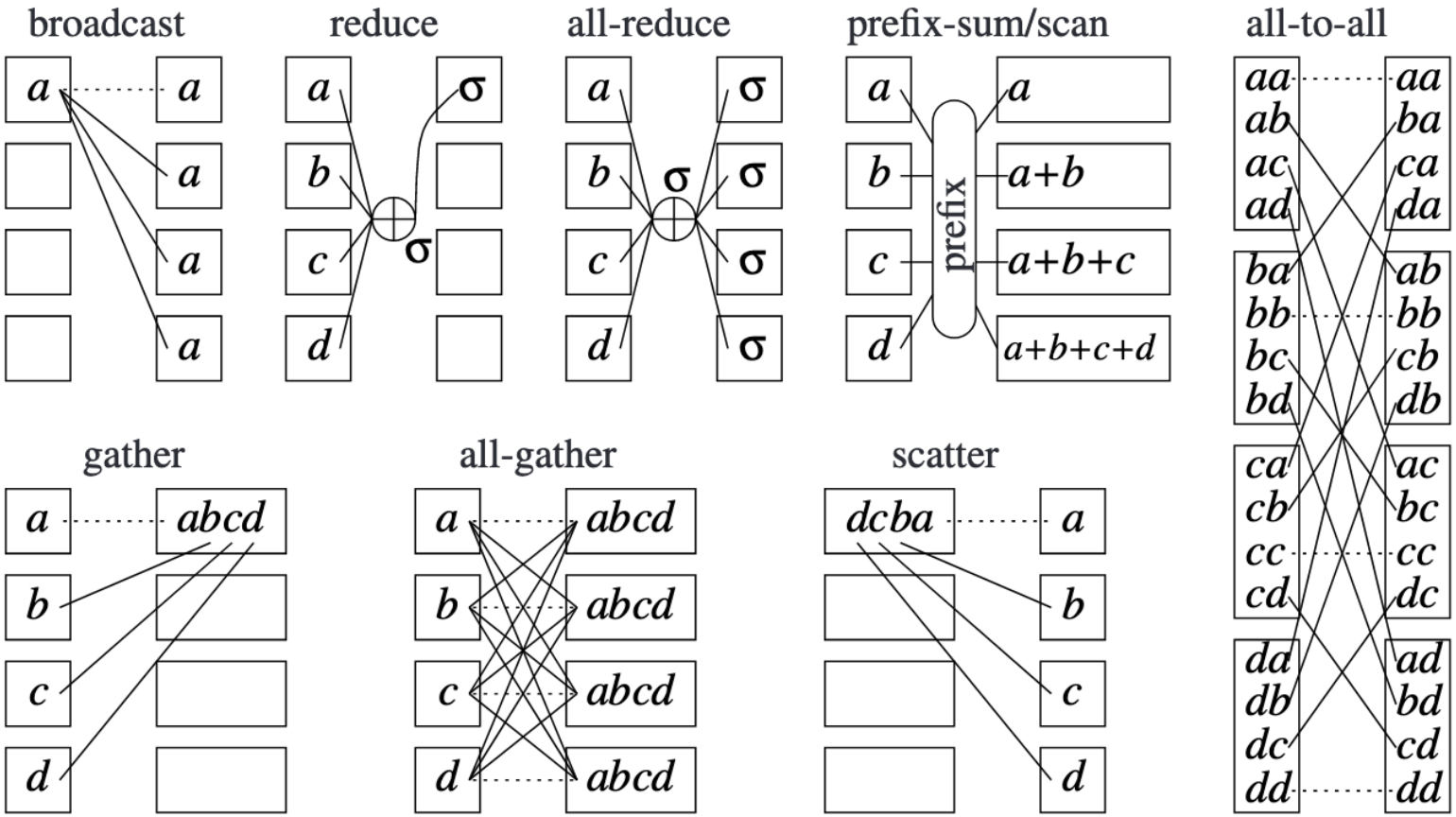



All-gather






Sequential and Parallel Algorithms and Data Structures: The Basic Toolbox.
Peter Sanders, Kurt Mehlhorn, Martin Dietzfelbinger, Roman Dementiev. 2019



NVIDIA

Big O, performance is in the constants

**Table 13.1.** Collective communication operations and their asymptotic complexities; $n$ = message size, $p$ = #PEs, $\alpha$ = message startup latency, $\beta$ = comm. cost per word

| Name | # senders | # receivers | # messages | Computations? | Complexity | Section |
|---|---|---|---|---|---|---|
| Broadcast | 1 | $p$ | 1 | no | $\alpha \log p + \beta n$ | 13.1 |
| Reduce | $p$ | 1 | $p$ | yes | $\alpha \log p + \beta n$ | 13.2 |
| All-reduce | $p$ | $p$ | $p$ | yes | $\alpha \log p + \beta n$ | 13.2 |
| Prefix sum | $p$ | $p$ | $p$ | yes | $\alpha \log p + \beta n$ | 13.3 |
| Barrier | $p$ | $p$ | 0 | no | $\alpha \log p$ | 13.4.2 |
| Gather | $p$ | 1 | $p$ | no | $\alpha \log p + \beta pn$ | 13.5 |
| All-gather | $p$ | $p$ | $p$ | no | $\alpha \log p + \beta pn$ | 13.5 |
| Scatter | 1 | $p$ | $p$ | no | $\alpha \log p + \beta pn$ | 13.5 |
| All-to-all | $p$ | $p$ | $p^2$ | no | $\log p(\alpha + \beta pn)$ or $p(\alpha + \beta n)$ | 13.6 |

Missing, reduce-scatter!!!

# Recursive Doubling for All-Gather

- Description: The recursive doubling algorithm enables efficient All-Gather by doubling the distance between nodes.
- Communication Pattern: In each round, a node communicates with another node at distance $2^i$, where $i$ increases with each round.
- Time Complexity: $O(\log(p)\alpha + pn\beta)$ .
- Message Complexity: $O(pn)$ . (Optimal)
- We it can use for reduce: $O\left(\log(p)\alpha + \frac{p-1}{p}n\beta\right)$
- Important: every round is a matching!
  - Overlay network
  - Butterfly topology
- $p$ is power of 2.

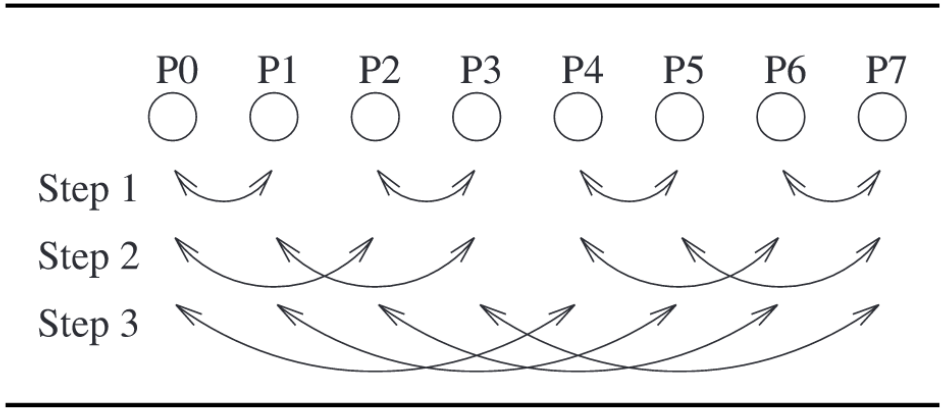



**Fig. 1 Recursive doubling for allgather.**

# Bruck Algorithm for All-Gather (any $p$)

- Description: The Bruck algorithm for All-Gather uses data reordering and minimizes communication steps.
- Communication Pattern: in each step $k$ $(0 \leq \ k < \ \lceil \log p \rceil)$, in each step k, process $i$ sends data to process $(i - 2^k)$ and receives data from process $(i + 2^k)$, (with wrap-around).
- Time Complexity: $O(\lceil \log p \rceil \alpha + \frac{p-1}{p} n\beta)$.
- Message Complexity: O($n$). (optimal)
- Can be used for All-reduce

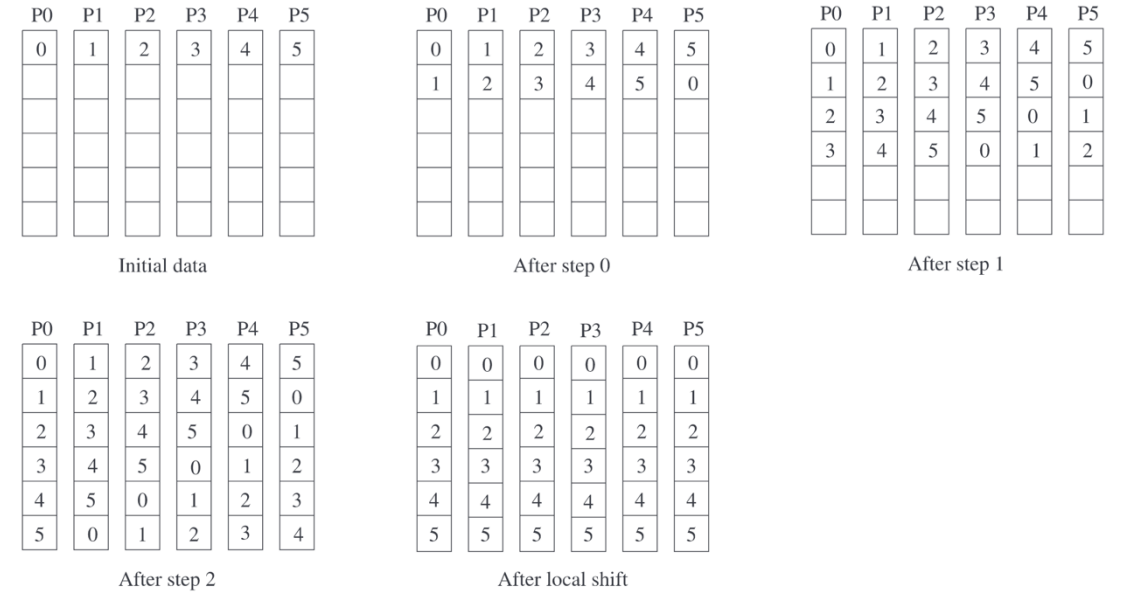



**Fig. 2 Bruck allgather.**

# Bruck Algorithm for All-2-All (any $p$)

- Description: The Bruck algorithm for All-Gather uses data reordering and minimizes communication steps.
- Communication Pattern: in each step $k$ $(0 \leq \ k < \ \lceil \log p \rceil)$, in each step k, process $i$ sends data to process $(i - 2^k)$ and receives data from process $(i + 2^k)$, (with wrap-around).
- Time Complexity: $O(\lceil \log p \rceil \alpha + \frac{(p-1)}{p} n\beta)$.
- Message Complexity: O($n$). (optimal)

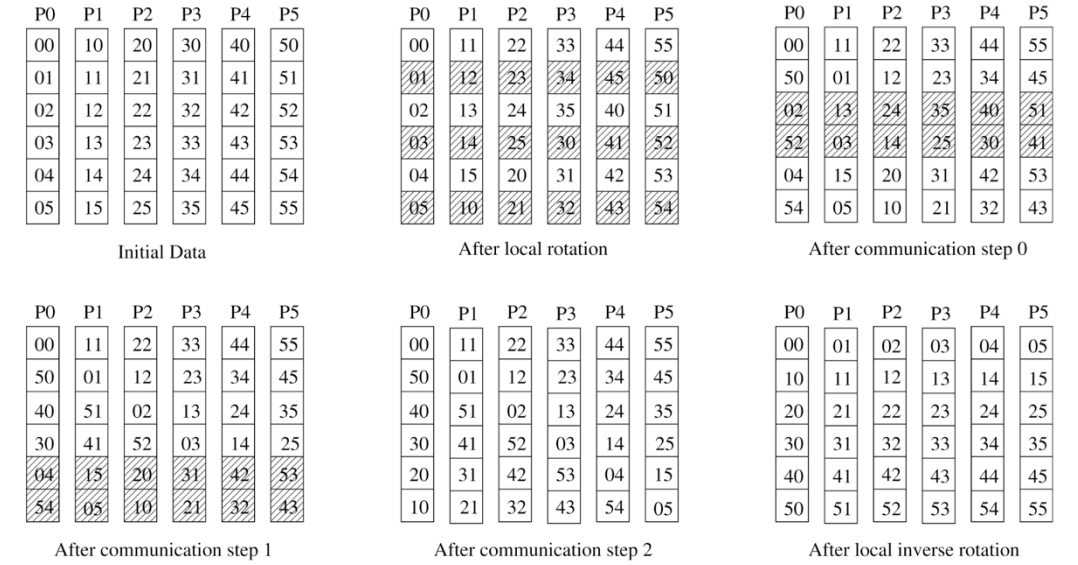



**Fig. 7 Bruck algorithm for all-to-all. The number *ij* in each box represents the data to be sent from process *i* to process *j*. The shaded boxes indicate the data to be communicated in the next step.**

# Recursive halving Algorithm

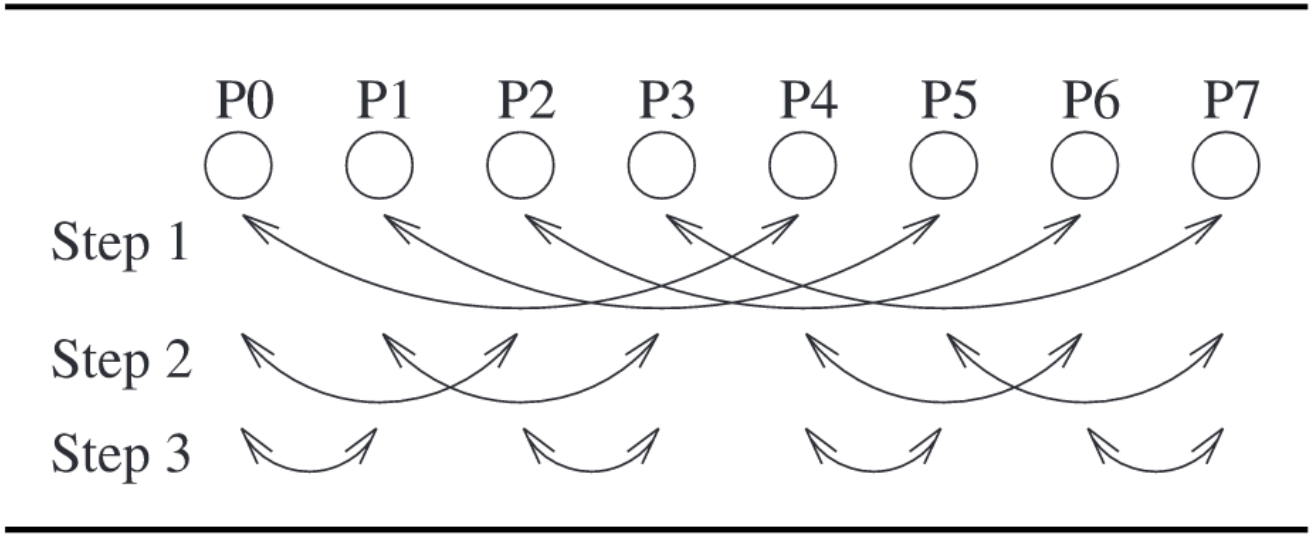
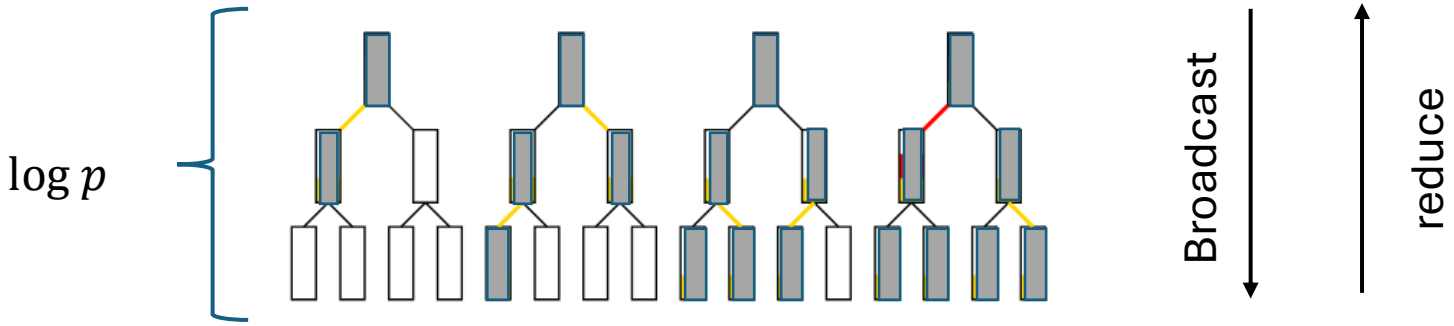
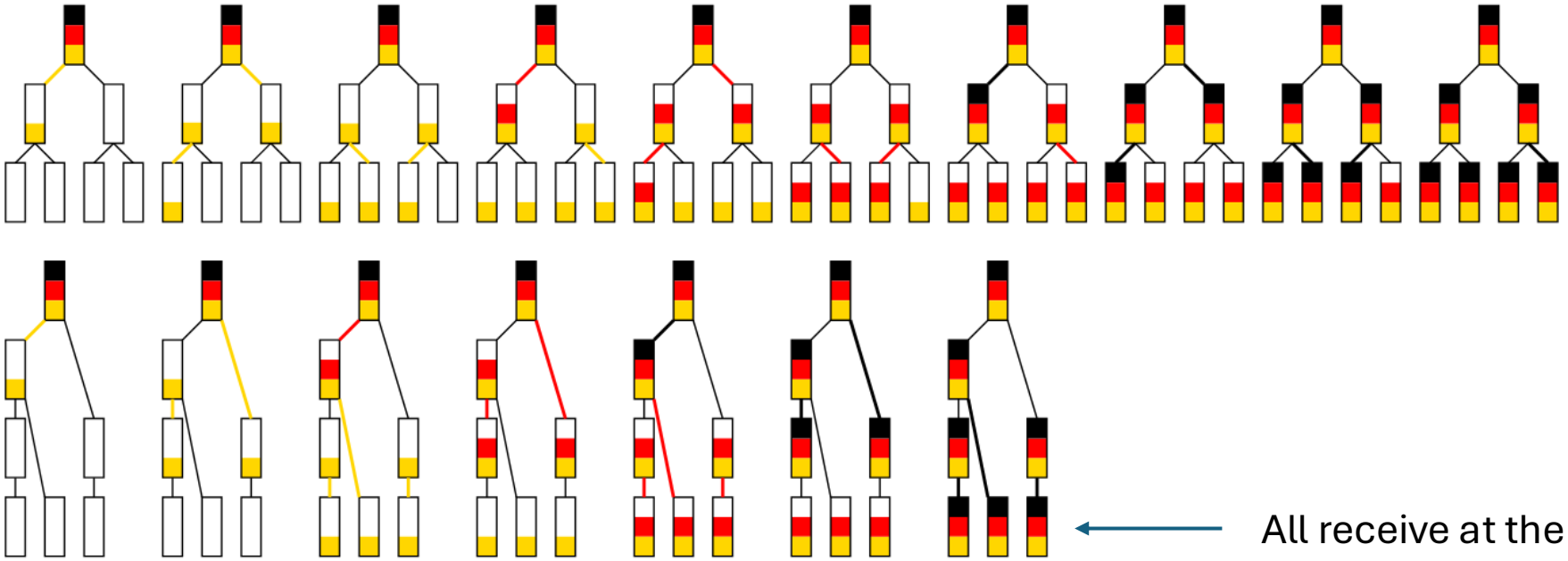



**Fig. 9 Recursive halving for commutative reduce-scatter.**




$$(\log p)(\alpha + \beta n)$$



All receive at the same time

**Fig. 13.4.** Pipelined binary tree broadcast with $p = 7$ and $k = 3$. Top: 10 steps using a complete binary tree and half-duplex communication. Bottom: 7 steps using a skewed tree and full-duplex communication.

$$T(L,n,k) \leq (2L + 3(k-1))\left(\alpha + \beta\left\lceil\frac{n}{k}\right\rceil\right), \quad \Longrightarrow \quad \mathrm{O}(\beta n + \alpha \log p)$$

Longest path $\approx \log p$ Number of packets

$$k = \sqrt{\beta n(2L - 3)/3\alpha}$$

Not always $p$

# New Scaling Algorithm and Initialization with NVIDIA Collective Communications Library 2.23

Jan 31, 2025

+5 Like Discuss (0)

## PAT logarithmic scaling for ReduceScatter and AllGather

The PAT algorithm is a variation of the Bruck algorithm, which features a logarithmic number of network steps for small sizes at scale, progressively increasing the number of network transfers as sizes increase, to keep buffering needs minimal. It applies to both AllGather and ReduceScatter. You can expect small to medium message sizes to perform better with PAT, with this improvement increasing as your workload scales.

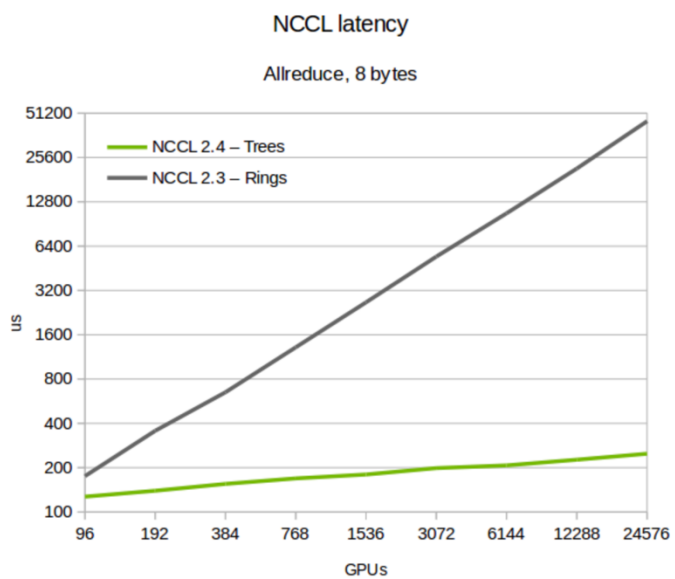



Figure 3. NCCL latency on up to 24,576 GPUs

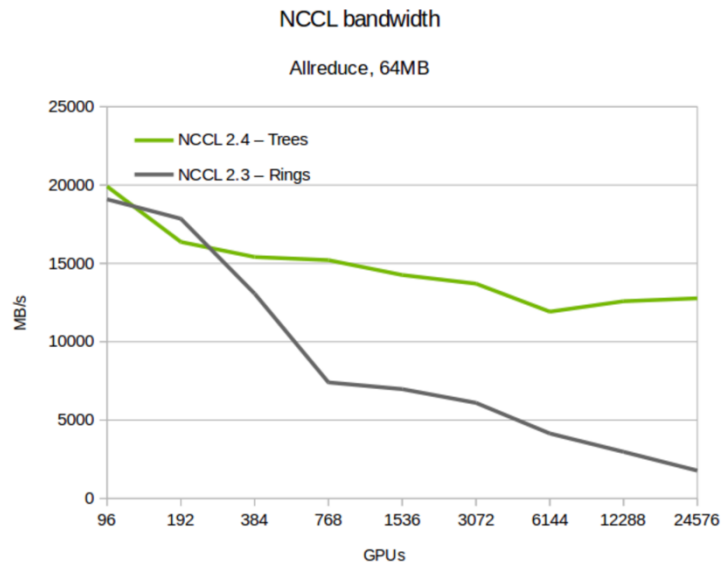



Figure 4. NCCL bus bandwidth on up to 24,576 GPUs

# Discussion

- Collectives emerged as the budling block of communication in distributed machine learning
- Many optimization opportunities:
  - Within collective: a lot of existing research
  - Overall communication patterns:
    - At large, current networks (Ethernet)  are not optimized for collective
    - Much less is done: routing, congestion, error recovery, topology, schedule
    - Better modeling for networking preformance
    - Failure is a larger issue: the whole collective may fail
- Questions?

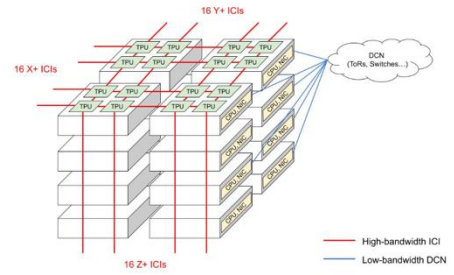



Figure 3: A 4x4x4 cube consists of 16 TPUv4 machines, each of which organizes 4 TPUs in a 2x2x1 mesh. The TPUs in a cube are interconnected over ICIs along *X*/*Y*/*Z* dimensions, with 16 optical links per cube face for OCS xconnect.